

Programación Orientada a Objetos II

Informe sobre Trabajo Práctico Integrador:

UNQ-Shop

Integrantes:

Dario Monteros (monterosrubendario@gmail.com)

Diego Arroyo (diegoramiroarroyo@gmail.com)

Ignacio Alvarez (nachoalvares@outlook.com)

Módulo 1 - Catálogo de productos

Patrón utilizado: Composite

Desde el inicio del trabajo, el equipo coincidió en que el módulo del catálogo encaja naturalmente con el patrón **Composite**. El enunciado establece que:

“El cliente que navega el catálogo no necesita saber si está consultando un producto simple o un paquete”.

Esto implica que productos y paquetes deben ser tratados de manera uniforme, lo cual es exactamente el problema que resuelve este patrón.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- Componente: Producto
- Compuesto: Paquete
- Hoja: ProductoIndividual

Decisiones de diseño

Inclusión de la interfaz ICatalogo

- Minuta de la reunión.
 - Tema a discutir la interfaz ICatalogo aporta valor o escalabilidad.

La conclusión fue :

- La interfaz **ICatalogo** no agrega complejidad innecesaria.

- Permite **polimorfismo** entre productos y paquetes.
- Evita condicionales del tipo instanceof.
- Facilita extender el catálogo con nuevos tipos de ítems sin romper código existente (**OCP**).

1. Producto y Paquete tienen responsabilidades claras.
2. Se pueden agregar nuevos tipos de ítems sin modificar código existente.
3. Paquete y Producto respetan el contrato de CatalogItem.
4. La interfaz es pequeña y específica.
5. El catálogo depende de la abstracción, no de implementaciones concretas.

Validación de atributos de los productos

En cuanto a la validación de atributos obligatorios y dinámicos, decidimos que un producto pueda auto validarse porque, tomando en cuenta el dominio propuesto por la consigna, consideramos que agregar una clase Validador sería complejizar innecesariamente el modelo. Todo esto porque, al fin y al cabo, para que la clase Validador no accediera a la lógica interna del Producto, tendría que definir un método parecido a:

```
Validador >> validarAtributosObligatoriosDe(Producto producto){

    return producto.validarAtributosObligatorios();

}
```

En vista de esto, preferimos simplificar y que la validación se acceda directamente desde el protocolo del producto.

Manejo de stock

Decidimos que el UNQ-Shop conozca sólo a un depósito porque consideramos que era suficiente para el modelo propuesto en la consigna. Si, en cambio, el Shop almacenara una lista de depósitos, las operaciones referidas al stock (incrementar, decrementar y preguntar si un producto en particular está en stock) serían mucho más costosas y, según nuestra opinión, excederían las dimensiones de este modelo.

En ningún momento el shop tiene conocimiento sobre la lógica relacionada al stock ya que, si bien su protocolo incluye mensajes sobre manejo de stock, la lógica sobre la validez de dichas operaciones se encuentra encapsulada en la clase Depósito. Esto minimiza el acoplamiento entre ambas clases.

Síntesis de las responsabilidades de cada clase

La clase UNQ-Shop respeta el principio de single responsibility porque solo se encarga de administrar operaciones relacionadas a los pedidos, delegando las operaciones relacionadas al stock a Depósito y la búsqueda al Catálogo. A su turno, la clase Depósito también respeta este principio porque sólo se encarga de las operaciones relacionadas al stock.

La clase catálogo contiene una sola instancia de cada producto disponible en el catálogo, de manera que funciona como una especie de vidriera de la que el cliente puede escoger. Como los profesores indicaron que no programáramos excepciones para este trabajo, si se intenta agregar un producto que ya está en el catálogo, no hace nada. En la misma línea, si se intenta eliminar un producto que no está en el catálogo tampoco hace nada.

Módulo 2 - Ciclo de vida del pedido

Patrón utilizado: State

Creímos conveniente aplicar este patrón a la implementación del ciclo de vida de los pedidos ya que la consigna especifica que, según el contexto, hay algunas transiciones que son válidas y otras que no. Esto coincide exactamente con uno de los casos de aplicación mencionados en la bibliografía, que recomienda el uso de este patrón cuando el comportamiento de un objeto depende de su estado y dicho objeto debe cambiar de comportamiento en tiempo de ejecución.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- Contexto: Pedido
- Estado: PedidoState
- EstadoConcreto: Borrador, Confirmado, EnPreparacion, Enviado, Entregado y Cancelado.

Módulo 3 - Métodos de envío

Patrón utilizado: Strategy

Aplicamos este patrón para los métodos de envío porque detectamos que el hecho de que el cálculo del costo y las condiciones varíen de acuerdo con la metodología elegida sugería que todos los métodos de envío en particular debían heredar este comportamiento de una clase para, posteriormente, implementarlo cada uno a su

manera. En otras palabras, muchas clases relacionadas (cada método de envío) se diferencian sólo en su comportamiento.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- Contexto: Pedido
- Estrategia: IEnvio
- EstrategiaConcreta: EnvioEstándar, EnvioExpress y RetiroEnSucursal

Esta solución respeta el principio open closed porque si se busca agregar otro método de envío, basta con sumar una clase que implemente la interfaz IEnvio.

Módulo 4 - Métodos de pago

Patrón utilizado: Template method

Se aplicó este patrón porque detectamos que todas las formas de pago compartían el mismo algoritmo. Por esto, la clase abstracta MetodoDePago define el método validarPago(monto), que contiene las operaciones primitivas para la validación de un pago (validar datos, reservar fondos, ejecutar la transacción y notificar el resultado). A su turno, cada subclase implementa cada uno de estos pasos de la manera que corresponda. De esta manera, el comportamiento común entre los métodos de pago está localizado en MetodoDePago, evitando la duplicación de código.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- ClaseAbstracta: MetodoDePago.
- ClaseConcreta: TarjetaDeCredito, TransferenciaBancaria y BilleteraVirtual.

Este modelo respeta el principio open-closed, ya que, si en el futuro se quiere añadir una nueva forma de pago, basta con incluir una nueva subclase de método de pago que implemente los pasos en la forma que corresponda.

Módulo 5 - Notificaciones del pedido

Patrón utilizado: Visitor

Al principio del proceso de diseño creímos conveniente utilizar el patrón Observer para implementar los subsistemas que deben reaccionar a los cambios de estado del pedido. Sin embargo, al ver que dicha reacción dependía de estados específicos

en cada implementación del subsistema, decidimos buscar otro patrón que nos permita contemplar cada caso por separado.

Como la consigna especifica que el módulo de pedidos no debe conocer ni depender de los distintos subsistemas que deben reaccionar ante los cambios de estado de un pedido, consideramos oportuno implementar el sistema de notificaciones haciendo uso del patrón visitor.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- Visitor: PedidoVisitor
- VisitorConcreto: NotificadorMail, GeneradorFactura y SistemaFidelizacion
- Elemento: PedidoState
- ElementoConcreto: Confirmado, Enviado, Entregado y Cancelado

Módulo 6 - Búsqueda en el catálogo

Patrón utilizado: Composite

La consigna da un indicio sobre la necesidad de hacer uso del patrón composite cuando precisa que existen criterios simples que pueden combinarse a través de operadores complejos que “pueden anidarse para construir consultas de cualquier profundidad”. Esto indica claramente la necesidad de que existan objetos compuestos que permitan realizar una recursión hasta llegar al valor buscado.

Según la definición propuesta por Gamma et. al, los roles de las clases con respecto a este patrón son:

- Componente: Criterio
- Compuesto: And, Or y Not
- Hoja: PorDisponibilidad, PorNombre, PorPrecioMaximo y PorCategoria

Esta solución respeta el principio open-closed porque si en el futuro se busca incluir un nuevo criterio, éste simplemente debe implementar la interfaz Criterio en función de su propia lógica. Además, respeta el single responsibility principle porque un criterio solo tiene la responsabilidad de saber si un producto lo cumple o no.

Decisiones de diseño

Se decidió incluir la clase abstracta CriterioCompuesto al detectar que tanto para el funcionamiento de And como de Or se necesitaban almacenar dos criterios. Si bien

somos conscientes de que dicha clase no es del todo necesaria, decidimos incluirla para evitar la repetición de código.

El criterio PorDisponibilidad conoce al shop ya que llegamos a la conclusión de que era la manera más limpia de preguntar por el stock. Esta alternativa evita la opción más débil de que un criterio conozca directamente al stock: en lugar de eso, PorDisponibilidad invoca el mensaje `shop.hayStockDe(IProducto)` mientras que, a su turno, el shop invoca el mensaje `this.deposito.hayStockDe(IProducto)`. Así, las responsabilidades quedan más claras y las clases no acceden a lógica que no tienen por que conocer. Además, para impedir que se modifique una instancia de UNQShop desde PorDisponibilidad, este criterio no tiene getters ni setters para el atributo shop.

Aclaración adicional

Queremos comunicarles que, debido a dificultades con otras materias y sumado a la cercanía de la fecha del parcial, no pudimos completar la totalidad del proyecto. Somos conscientes de que el trabajo no cumple con el 95% de la cobertura de tests de unidad sobre el código del dominio y de que el módulo de reportes no está implementado. Sin embargo, vamos a procurar que esté completo para la segunda fecha de entrega.